KẾ HOẠCH THỰC HIỆN DỰ ÁN

HỆ THỐNG CHẤM BÀI KIỂM TRA BẰNG OCR, RAG VÀ LLM

Phạm vi: câu trả lời dạng văn bản; không xử lý biểu thức toán học

```
Exam Image
    ↓
Preprocessing → Line Segmentation → OCR → Reconstruction → exam.md
                                                              ↓
Reference Documents → RAG → Generated Rubric → LLM Grading
                                                              ↓
                                                    Score + Feedback
```

Tài liệu kế hoạch triển khai

1. Mục tiêu và phạm vi

Mục tiêu của dự án là xây dựng một pipeline có khả năng nhận ảnh bài kiểm tra, chuyển toàn bộ nội dung chữ trên trang thành văn bản có cấu trúc, truy xuất kiến thức tham khảo và dùng LLM để sinh tiêu chí chấm cũng như đánh giá câu trả lời của người học.

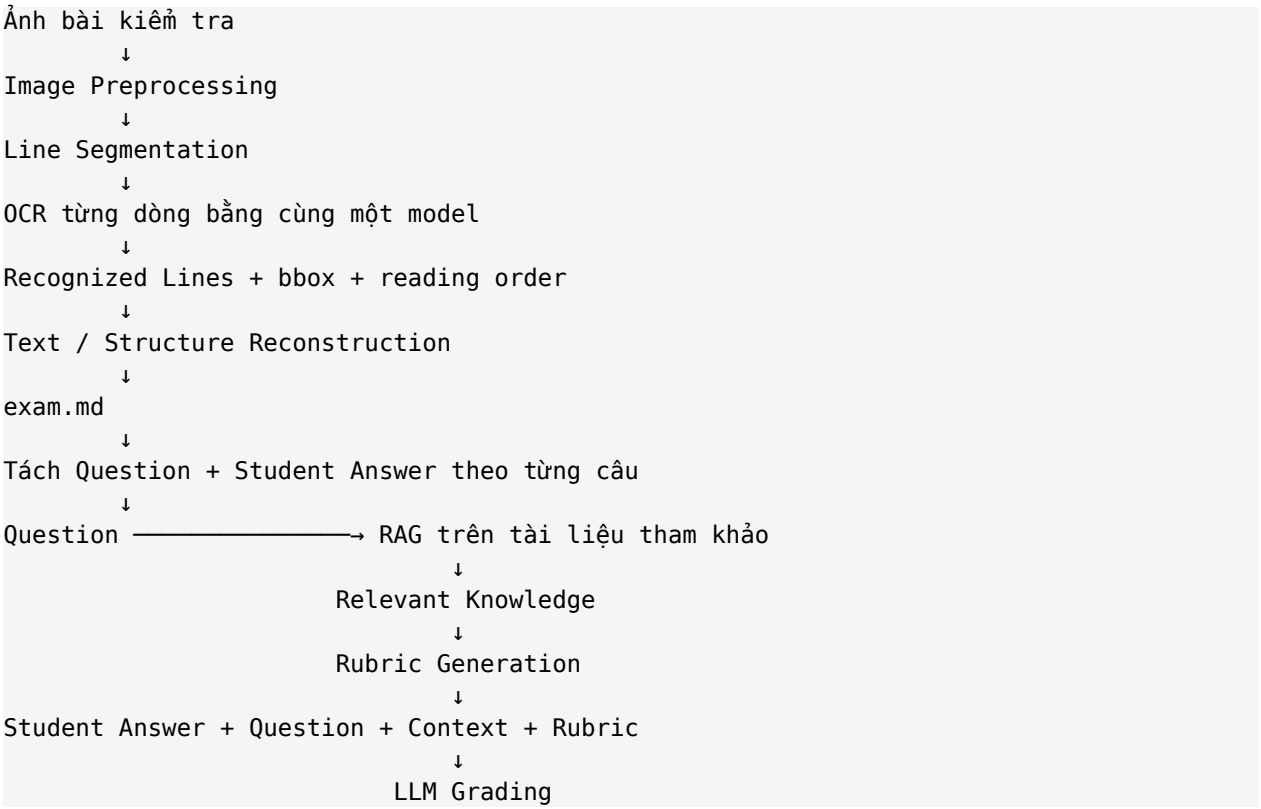
1.1. Mục tiêu chức năng

- Nhận ảnh bài kiểm tra từ ảnh chụp hoặc ảnh scan.
- Tiền xử lý ảnh để ổn định chất lượng đầu vào.
- Phân đoạn toàn bộ trang thành các dòng chữ theo thứ tự đọc.
- Dùng một mô hình OCR duy nhất để nhận dạng tất cả dòng chữ, không tách riêng OCR đề bài và OCR câu trả lời.
- Ghép kết quả OCR thành file exam.md có cấu trúc câu hỏi và câu trả lời.
- Xây dựng kho tài liệu tham khảo và RAG để truy xuất kiến thức liên quan theo từng câu hỏi.
- Trong trường hợp không có rubric sẵn, dùng câu hỏi và kiến thức truy xuất để sinh rubric/tiêu chí chấm.
- Dùng LLM chấm từng câu và sinh điểm, phản hồi, dẫn chứng.
- Đánh giá riêng từng tầng: OCR, retrieval và grading.

1.2. Ngoài phạm vi

- Không nhận dạng công thức toán học hoặc biểu thức viết tay.
- Không chuyển công thức sang LaTeX.
- Không xây dựng hệ thống chấm toán tự động.
- Không bắt buộc triển khai ứng dụng web/mobile trong giai đoạn đầu.
- Không dùng VLM để bỏ qua hoàn toàn OCR; OCR vẫn là một thành phần nghiên cứu chính.

2. Pipeline tổng thể



```
                                ↓
                        Score + Feedback
```

Điểm kiến trúc quan trọng: OCR không cần biết một dòng là chữ in hay chữ viết tay, cũng không cần có hai pipeline khác nhau. Mỗi dòng chữ được đưa qua cùng một recognizer. Việc xác định cấu trúc câu hỏi - câu trả lời được thực hiện sau OCR trong bước reconstruction.

3. Các module và hợp đồng dữ liệu

Module	Input	Output	Vai trò
Preprocessing	Ảnh gốc	Ảnh đã chuẩn hóa	Cải thiện chất lượng cho segmentation/OCR
Line Segmentation	Ảnh đã xử lý	Line images + bbox + reading order	Tách toàn bộ dòng chữ
OCR	Line image	Recognized text	CRNN + BiLSTM + CTC baseline
Reconstruction	Text + bbox + order	exam.md	Khôi phục thứ tự và cấu trúc
RAG	Question + knowledge base	Relevant chunks	Truy xuất kiến thức theo câu
Rubric Generation	Question + context	rubric.json	Sinh tiêu chí chấm nếu chưa có rubric
LLM Grading	Question + answer + context + rubric	grading.json + feedback.md	Chấm điểm có cấu trúc

4. Pipeline OCR

4.1. Tiền xử lý ảnh

- Grayscale khi phù hợp.
- Denoise để giảm nhiễu ảnh chụp/scan.
- Deskew để sửa ảnh bị nghiêng.
- Normalize/resize nhưng giữ tỷ lệ hợp lý.
- Threshold/binarization chỉ dùng khi thực nghiệm cho thấy có lợi.

Không áp dụng mọi phép biến đổi một cách cố định. Các bước tiền xử lý cần có cấu hình để so sánh tác động tới CER/WER.

4.2. Phân đoạn dòng

Mục tiêu của segmentation là tạo ra các ảnh dòng độc lập và giữ metadata vị trí. Ở giai đoạn này không cần xác định dòng nào là câu hỏi hay câu trả lời.

```
{
  "line_id": 12,
  "page": 1,
  "bbox": [100, 420, 1250, 490],
  "reading_order": 12,
  "image_path": "data/line_images/page1_line12.png"
}
```

4.3. Mô hình OCR baseline

```
Line Image
    ↓
```

```
CNN Feature Extractor
    ↓
Sequence Features
    ↓
Bidirectional LSTM
    ↓
Character Logits
    ↓
CTC Decode
    ↓
Vietnamese Text
```

Baseline ưu tiên là CRNN + BiLSTM + CTC vì nhẹ, dễ fine-tune trên Google Colab, phù hợp với OCR theo dòng và thuận tiện để đo CER/WER. Một model duy nhất được dùng cho toàn bộ dòng chữ của trang.

4.4. Train và inference

```
Google Colab
    ↓
Train / Fine-tune OCR
    ↓
Checkpoint
    ↓
Download checkpoint
    ↓
Local inference hoặc Colab inference
```

Training và inference phải tách rời. Checkpoint đã train phải load được trong code inference mà không phụ thuộc toàn bộ notebook training.

5. Reconstruction và exam.md

Sau OCR, hệ thống dùng text nhận dạng cùng bbox và reading_order để ghép lại nội dung. Kết quả trung gian chính của toàn bộ phần OCR là exam.md.

```
# Exam

## Question 1

### Question
Trình bày khái niệm Machine Learning.

### Student Answer
Machine Learning là một nhánh của trí tuệ nhân tạo...

## Question 2

### Question
Phân biệt supervised learning và unsupervised learning.

### Student Answer
...
```

Nếu reconstruction chưa xác định hoàn toàn cấu trúc Question/Answer ở giai đoạn đầu, có thể giữ text theo thứ tự dòng trước, sau đó bổ sung parser dựa trên số câu, layout, khoảng cách hoặc quy tắc định dạng. Mục tiêu cuối cùng vẫn là tạo cấu trúc có thể tách từng cặp Question - Student Answer.

6. Knowledge Base và RAG

6.1. Nguồn tài liệu

- Slide môn học.
- Lecture notes/giáo trình.
- Tài liệu do giảng viên cung cấp.
- Đáp án tham khảo nếu có.
- Rubric có sẵn nếu có.

Student answer không được đưa vào vector database như nguồn kiến thức chuẩn.

6.2. Ingestion

```
Reference Documents
        ↓
Text Extraction
        ↓
Cleaning
        ↓
Chunking
        ↓
Embedding
        ↓
Local Vector Store
```

6.3. Retrieval

```
Question
   ↓
Retrieval Query
   ↓
Vector Search
   ↓
Top-k Relevant Chunks
```

Question là tín hiệu truy xuất chính. Có thể bổ sung Student Answer vào query trong một số thí nghiệm, nhưng không nên chỉ dựa vào Student Answer vì bài làm sai có thể làm lệch retrieval.

7. Sinh rubric khi chưa có tiêu chí chấm

Nếu chỉ có tài liệu tham khảo mà chưa có rubric, hệ thống tạo rubric trước khi chấm bài. Rubric được sinh từ Question và Retrieved Knowledge, không phụ thuộc nội dung câu trả lời của một sinh viên cụ thể.

```
Question + Retrieved Knowledge
            ↓
      LLM Rubric Generator
            ↓
        rubric.json
```

```
{
  "question_id": "Q1",
  "max_score": 5,
  "criteria": [
    {"criterion": "Nêu đúng khái niệm chính", "points": 2},
    {"criterion": "Giải thích cơ chế/ý nghĩa", "points": 2},
    {"criterion": "Diễn đạt đúng bản chất", "points": 1}
  ]
}
```

Với cùng một câu hỏi, rubric nên được lưu lại và dùng chung để chấm nhiều bài, giúp tiêu chí ổn định hơn.

8. LLM Grading

```
Question
+ Student Answer
+ Retrieved Knowledge
+ Rubric
        ↓
    LLM Grader
        ↓
Score + Feedback + Evidence
```

Grader phải trả kết quả có cấu trúc để dễ đánh giá và tổng hợp.

```
{
  "question_id": "Q1",
  "score": 4.0,
  "max_score": 5.0,
  "feedback": "Đúng ý chính nhưng còn thiếu một luận điểm.",
  "evidence": ["Đoạn tài liệu liên quan..."]
}
```

- Chấm theo rubric đã tạo/được cung cấp.
- Dùng evidence truy xuất từ tài liệu.
- Không tự thêm tiêu chí mới trong lúc chấm.
- Có thể bỏ qua lỗi OCR nhỏ nếu ý nghĩa câu trả lời vẫn rõ.
- Phân biệt lỗi nhận dạng OCR và lỗi kiến thức nếu có đủ bằng chứng.
- Không vượt quá max_score.

9. Cấu trúc repository đề xuất

```
handwritten-exam-grading/
├── README.md
├── requirements.txt
├── .env.example
├── .gitignore
│
├── notebooks/
│   ├── ocr_training.ipynb
│   └── ocr_experiments.ipynb
│
├── src/
│   ├── preprocessing/
│   │   └── preprocess.py
│   ├── segmentation/
│   │   └── line_segmentation.py
│   ├── ocr/
│   │   ├── model.py
│   │   ├── charset.py
│   │   ├── decoder.py
│   │   ├── inference.py
│   │   └── metrics.py
│   ├── reconstruction/
│   │   ├── reading_order.py
│   │   ├── structure_parser.py
│   │   └── markdown_writer.py
│   ├── rag/
│   │   ├── loaders.py
│   │   ├── chunking.py
│   │   ├── embeddings.py
```

```
│   │   ├── vector_store.py
│   │   └── retriever.py
│   ├── rubric/
│   │   └── generator.py
│   ├── grading/
│   │   ├── llm_client.py
│   │   ├── prompts.py
│   │   ├── schemas.py
│   │   └── grader.py
│   └── pipeline/
│       ├── ocr_pipeline.py
│       ├── rag_pipeline.py
│       └── grading_pipeline.py
│
├── data/
│   ├── raw/exams/
│   ├── processed/
│   └── line_images/
│
├── knowledge/
│   ├── documents/
│   ├── reference_answers/
│   └── generated_rubrics/
│
├── models/
│   ├── checkpoints/
│   └── charset.txt
│
├── outputs/
│   ├── exam.md
│   ├── grading.json
│   └── feedback.md
│
└── tests/
    ├── test_preprocessing.py
    ├── test_segmentation.py
    ├── test_ocr.py
    ├── test_reconstruction.py
    ├── test_rag.py
    └── test_grading.py
```

10. Kế hoạch triển khai theo giai đoạn

Giai đoạn	Công việc chính	Đầu ra/Done
Phase 1 - Nền tảng repo	Khởi tạo cấu trúc thư mục, config, dependency, sample data, README ban đầu.	Repo chạy được; có sample input/output.
Phase 2 - OCR baseline	Xây CRNN + BiLSTM + CTC, charset, decoder, dataset loader, CER/WER.	Nhận dạng được ảnh một dòng và có checkpoint.
Phase 3 - Colab training	Notebook train/fine-tune, save/load checkpoint, theo dõi loss và metric.	Train được trên Colab và tải model về dùng inference.
Phase 4 - Line segmentation	Tiền xử lý trang, tách line, lưu bbox và reading order.	Ảnh bài kiểm tra được tách thành line images đúng thứ tự.

Giai đoạn	Công việc chính	Đầu ra/Done
Phase 5 - Reconstruction	Ghép kết quả OCR, parser cấu trúc, sinh exam.md.	Có exam.md chứa Question/Student Answer theo từng câu.
Phase 6 - RAG	Loader, chunking, embeddings, vector store, retriever.	Truy xuất được top-k đoạn phù hợp với từng câu hỏi.
Phase 7 - Rubric generation	Sinh rubric từ Question + context và lưu lại.	Có rubric JSON ổn định cho từng câu.
Phase 8 - LLM grading	Prompt, schema, grader, feedback và tổng điểm.	Có grading.json và feedback.md.
Phase 9 - Integration	Nối OCR → exam.md → RAG → rubric → grading.	Chạy được end-to-end trên bộ mẫu.
Phase 10 - Evaluation	So sánh OCR, retrieval, grading; phân tích lỗi.	Có bảng metric và kết luận thực nghiệm.

11. Tiêu chí đánh giá

Tầng	Metric	Ý nghĩa
OCR	CER, WER	So sánh prediction với ground truth text.
OCR Runtime	Model size, inference time	Đánh giá khả năng chạy local.
Retrieval	Recall@k / Precision@k khi có nhãn	Đánh giá context được truy xuất có đúng không.
Grading	MAE, agreement, correlation với điểm giáo viên	Cần tập dữ liệu có điểm tham chiếu.
End-to-end	Error analysis	Tách nguyên nhân lỗi: segmentation, OCR, retrieval hay grading.

11.1. Thí nghiệm nên có

1. So sánh preprocessing khác nhau trên cùng OCR model.
2. So sánh CRNN baseline với ít nhất một biến thể cải tiến nếu đủ thời gian.
3. So sánh LLM grading không RAG với RAG + LLM grading.
4. So sánh RAG + LLM grading với RAG + generated rubric + LLM grading.
5. Phân tích trường hợp điểm sai do OCR và trường hợp sai do reasoning/grading.

12. Rủi ro và hướng xử lý

Rủi ro	Hướng xử lý
Chữ viết tay đa dạng	Fine-tune trên dữ liệu gần domain; augmentation; đánh giá theo người viết.
Segmentation sai dòng	Giữ bbox để debug; test trên nhiều layout; cho phép fallback/manual template ban đầu.
Model OCR đọc chữ in/chữ viết tay không đồng đều	Trộn dữ liệu phù hợp khi train/fine-tune; đánh giá riêng theo

Rủi ro	Hướng xử lý
	loại chữ nếu cần.
exam.md sai cấu trúc	Tách reconstruction thành module riêng; lưu intermediate line JSON để tái tạo.
RAG lấy sai context	Điều chỉnh chunking, embedding, top-k; retrieval theo Question.
Rubric sinh thiếu/không ổn định	Sinh rubric một lần cho mỗi câu; lưu version; review rubric ở giai đoạn thử nghiệm.
LLM chấm không nhất quán	Structured prompt + fixed rubric + temperature thấp + schema validation.
Chi phí API	Dùng RAG local; cache rubric/context; chỉ gọi LLM ở bước grading cần thiết.

13. Definition of Done - phiên bản tối thiểu

6. Nhận một ảnh bài kiểm tra mẫu.
7. Tách được các dòng chữ và lưu metadata thứ tự đọc.
8. OCR toàn bộ dòng bằng cùng một model.
9. Ghép kết quả thành exam.md.
10. Tách được từng Question và Student Answer từ exam.md.
11. Index được tập tài liệu tham khảo.
12. Retrieve được context theo từng câu hỏi.
13. Sinh được rubric khi không có rubric sẵn.
14. LLM chấm và trả grading.json có schema rõ ràng.
15. Sinh feedback.md cho người dùng.
16. Có CER/WER cho OCR và ít nhất một đánh giá định lượng/định tính cho grading.
17. Pipeline có thể chạy lại từ các lệnh được mô tả trong README.

14. Kết luận kiến trúc đã chốt

```
ONE OCR PIPELINE
Exam Image
→ Preprocessing
→ Line Segmentation
→ Same OCR Model
→ Reconstruction
→ exam.md

KNOWLEDGE PIPELINE
Reference Documents
→ Chunking
→ Embedding
→ Vector Store
→ Retrieval

GRADING PIPELINE
Question + Retrieved Knowledge
→ Generate Rubric
→ Student Answer + Rubric + Context
→ LLM Grading
→ Score + Feedback
```

Kiến trúc này giữ dự án đủ gọn để triển khai theo môn học nhưng vẫn có các phần nghiên cứu rõ ràng: OCR chữ viết tay, reconstruction, retrieval và LLM grading. Math OCR được loại khỏi phạm vi; toàn bộ chữ trên bài được xử lý bởi một OCR pipeline thống nhất.